```
button = document.getElementById("somebutton");
button.addEventListener("click",hide);
function hide(event)
{
document.getElementById("somediv").
setStyle("display","none");
}
</script>
```

To find out who fired the event, use the event.target.getId() method of an event object.

```
function hide(event)
{
document.getElementById("somediv").
setStyle("display","none");
usedby = event.target.getId();
new Dialog().showMessage("Info","This event was used by
"+usedby+"","Okay")
}
```

When you attach an event to a Data Object Model node, the event gets automatically added to all the child nodes in that node. So just which object is invoking the event listener, and then take action using getId().

There are other useful methods available to this event object, which are essential in developing sophisticated event-based applications:

- `stopPropagation()`—Stop the propagation of the event further to any DOM sub- node.
- `cancelDefault()`—Cancel the default behaviour of an event.
- `listEventListeners(eventName)`—Return an array of handles of the events which have been added to this event. It also includes the events that were added in FBML using the <event> attribute.
- `purgeEventListeners(eventName)`—Remove all event listeners for this event.

Example:

```
<div style="padding:20px">
<textarea id="sometextarea">
</textarea>
</div>
<script>
ta = document.getElementById("sometextarea");
```